

Sharing KV Caches Across Fine-Tuned Language Models with a Head-Wise Linear Map

Mohammad Shahid Chaudhary
Independent Researcher
shahidchaudhary9@protonmail.com

Abstract

Fine-tuning language models from a shared base causes their internal key-value (KV) representations to drift, which prevents one model from attending over another’s cache. We show that at GPT-2 scale this drift is largely linear. We introduce W_{map} , a head-wise linear projection that maps keys and values between two fine-tuned models ($K_B \approx K_A W_k$, $V_B \approx V_A W_v$), and we show that the projection need never be applied to the cache itself. Because the map is linear and acts within each head, the associativity of matrix multiplication lets it be folded into the incoming query and the attention output instead, which reduces the per-step conversion cost from $O(S)$ to $O(1)$ in the context length S and leaves the stored cache untouched. On WikiText-2 the consuming model reproduces 93.4% of its own native token choices, and 99.8% of those it makes with a decisive margin, while storing no cache of its own. Between two independently fine-tuned domain specialists, the map roughly doubles the number of tokens a free autoregressive rollout survives before diverging from the native trajectory.

1 Introduction

Serving several fine-tuned variants of a single base model has become a common deployment pattern. A router may dispatch each request to a specialist, or a pipeline may hand a conversation from one specialist to another partway through. In both cases the KV cache is duplicated: every model encodes the same prefix into its own cache, and a model that inherits a context from elsewhere must re-prefill it before it can generate. At fp16 a 16k-token context in GPT-2 occupies roughly 576 MB per model, so the duplication is not a minor overhead.

The caches cannot simply be reused because fine-tuning moves the key and value geometry. Two specialists derived from one checkpoint still share a tokenizer, a depth, a head count and an attention layout, but their cached tensors are no longer interchangeable: handing one model the other’s cache produces degraded and, under free generation, rapidly diverging output. We ask how much of that displacement can be undone by a linear map, and find that at GPT-2 scale most of it can. A bias-free 64×64 matrix per head per layer, totalling about 1% of the backbone parameters, recovers most of the consuming model’s behaviour from a cache it did not produce.

Used naively, such a map would not help. Multiplying an S -token cache by W_k and W_v at each decoding step costs $O(Sd^2)$ per layer, which grows with the context and defeats the purpose of not re-prefilling. Since the map is linear and confined to a single head, it can instead be applied to the incoming query and to the attention output, at a cost of $O(d^2)$ independent of S . This is the same algebraic move that multi-head latent attention uses to avoid materializing keys and values from its compressed cache [10], applied here to translation between models rather than to decompression within one.

We make three contributions. First, we describe and train W_{map} , a head-wise bias-free linear adapter that translates KV caches between models fine-tuned from a shared base, using output-space distillation together with an activation alignment term. Second, we give the reformulation that turns the projection from an $O(S)$ cache rewrite into an $O(1)$ query and output transform, and verify that the two orders of computation agree to the last bit in single precision. Third, we measure the result along four axes: task retention under a fully shared cache, decode latency and memory scaling from 128 to 16,384 tokens, full-vocabulary generative fidelity on WikiText-2, and transfer between two domain specialists under free autoregressive rollout.

2 Related Work

Most work on KV cache cost operates within a single model. The cache is well established as the binding constraint on long-context decoding [3]. Multi-query and grouped-query attention reduce the number of distinct key and value heads [5, 6], cross-layer attention shares them between adjacent layers [7], and multi-head latent attention stores a compressed latent from which keys and values are never explicitly reconstructed [10]. A second line evicts or compresses entries after the fact, selecting which tokens to retain by accumulated attention mass [8] or by keeping an initial sink together with a sliding window [9]. A third reuses cache across requests that share text: paged allocation with copy-on-write [4], precomputed modules for recurring prompt segments [11], and prefix trees over a running workload [12]. All of these share cache within one set of weights. The question here is different: whether cache produced under one set of weights can be consumed under another.

That question connects to work on comparing and connecting learned representations. Linear maps have long been used to align independently trained embedding spaces [15], and model stitching inserts a trainable layer between the lower half of one network and the upper half of another to test whether their intermediate representations are compatible [16, 17]. Representational similarity measures such as CKA quantify the same relationship without constructing a working hybrid [18], while relative representations achieve zero-shot communication between latent spaces by re-expressing points relative to shared anchors [19]. The broader observation that independently trained models converge on similar representations [20], and that fine-tuning deltas behave additively in weight space [21], both suggest that the gap between two fine-tunes of one checkpoint should be small and structured. Our adapter is a stitching layer applied at every attention head rather than at one cut point, and applied to cached activations rather than to a forward pass. In the sense that it adds a small trained matrix to a frozen backbone it resembles parameter-efficient adaptation such as LoRA [13], but it modifies no forward computation of either model and exists only to translate between them.

Folding a linear map into neighbouring weights is itself standard. DeepSeek-V2 absorbs the key up-projection into the query projection and the value up-projection into the output projection, for exactly the reason we do: it removes an $O(S)$ per-step transform of the cache [10]. That work also notes that rotary embeddings [26] block the absorption when the rotation is interposed between the two matrices, a point we return to in Section 5.

3 Method

3.1 Setup and notation

All experiments use GPT-2 small [2] (124M), a standard decoder-only transformer [1] with $L = 12$ layers, $H = 12$ heads, head dimension $d = 64$, hidden size 768 and a vocabulary of 50,257. Specialists

are produced by full fine-tuning from the same public checkpoint, so they agree on tokenizer, depth and attention layout but not on weights.

We write A for the producer, the model whose forward pass created the cache, and B for the consumer, the model that must attend over it. For layer l and head h , let $K_A^{(l,h)}, V_A^{(l,h)} \in \mathbb{R}^{S \times d}$ be the cached keys and values over S prefix tokens. We seek matrices that place them in B 's space:

$$\hat{K}_B^{(l,h)} = K_A^{(l,h)} W_k^{(l,h)}, \quad \hat{V}_B^{(l,h)} = V_A^{(l,h)} W_v^{(l,h)}, \quad W_k^{(l,h)}, W_v^{(l,h)} \in \mathbb{R}^{d \times d}.$$

3.2 The map

W_{map} is the collection $\{W_k^{(l,h)}, W_v^{(l,h)}\}$ together with gain vectors $\gamma_k^{(l,h)}, \gamma_v^{(l,h)} \in \mathbb{R}^d$ applied element-wise after the multiplication. The gains exist only to help optimization; because they scale output coordinates they can be folded back into the matrices as $W \leftarrow W \text{diag}(\gamma)$, so the effective operator is a single linear map. Matrices are initialized to the identity and gains to one, which makes the untrained adapter a no-op.

The map is block-diagonal across heads, so head h of A maps only to head h of B , and it carries no bias term. Neither choice is incidental: a bias or a cross-head block would invalidate the reformulation in Section 3.3, since the argument depends on the transform acting on the feature axis within the same head that the softmax operates over.

The parameter count is $2LHd^2 = 1,179,648$ matrix entries plus $2LHd = 18,432$ gains, about 1.20M parameters in total, or 0.97% of the 124M backbone. One adapter is required per ordered model pair.

3.3 Constant-time decoding

During decoding, a single query $Q \in \mathbb{R}^{1 \times d}$ attends over the projected cache. Because W_k acts on the feature axis of K , we have $Q(KW_k)^\top = QW_k^\top K^\top$, so the attention scores, and hence the softmax weights, are unchanged if the query is projected instead of the keys. The value projection commutes with the convex combination the softmax performs. Writing $\tilde{Q} = QW_k^\top$,

$$\text{Attn}(Q, KW_k, VW_v) = \text{softmax}\left(\frac{\tilde{Q}K^\top}{\sqrt{d}}\right) V W_v = \text{Attn}(\tilde{Q}, K, V) W_v. \quad (1)$$

The right-hand side never reads or writes the cache. Per layer per decoding step the left-hand side costs $2Sd^2$ multiply-accumulates for the two cache projections, while the right-hand side costs $2d^2$ for one query projection and one output projection, so the conversion overhead no longer depends on the context length. A further consequence is that the cache can remain in A 's space and be read concurrently by any number of consumers, each applying its own W_{map} , with no additional storage per consumer. Equation (1) is an identity rather than an approximation; we confirm this numerically in Section 4.2.

3.4 Training

Each W_{map} is trained with the base model and both specialists frozen, so only the adapter receives gradients. For each training sequence we take a 128-token prefix, run the producer over it, project the resulting keys and values, and have the consumer predict the next 64 tokens conditioned entirely

on the projected cache. The loss combines an output-space term with a representation-space term,

$$\mathcal{L} = \underbrace{T^2 D_{\text{KL}}(P_B \| P_{\hat{B}})}_{\text{distillation}} + \lambda \cdot \underbrace{\frac{1}{L} \sum_{l=1}^L [\text{MSE}(\hat{K}^{(l)}, K_B^{(l)}) + \text{MSE}(\hat{V}^{(l)}, V_B^{(l)}) + 2(d_{\cos}(\hat{K}^{(l)}, K_B^{(l)}) + d_{\cos}(\hat{V}^{(l)}, V_B^{(l)}))]}_{\text{activation alignment}}, \quad (2)$$

with $T = 2$ and $\lambda = 0.5$, following standard logit distillation [14]. Here P_B is the consumer’s next-token distribution over the full 50,257-token vocabulary when it runs natively on the whole sequence, $P_{\hat{B}}$ is its distribution when reading the projected cache, and $d_{\cos} = 1 - \cos$. The distillation term drives downstream behaviour; the alignment term constrains the intermediate geometry so that the map does not reach solutions which match logits while placing the keys and values somewhere unrelated.

We optimize with AdamW at a peak learning rate of 10^{-3} , weight decay 10^{-4} on the matrices and none on the gains, gradient accumulation of 4, cosine annealing to 10^{-5} , gradient clipping at 1.0, and 5 epochs over 1000 sequences. The lowest-loss epoch is retained. All runs use seed 42.

3.5 Saliency-guided mixed cache

In the pure setting the consumer stores nothing. As an intermediate point we also retain a small fraction of its native entries. For prefix position i we compute

$$s_i = \left(\sum_{l,h} \alpha_i^{(l,h)} \right) \cdot \left(\sum_l \|\hat{K}_i^{(l)} - K_{B,i}^{(l)}\|^2 + \|\hat{V}_i^{(l)} - V_{B,i}^{(l)}\|^2 \right),$$

the attention mass the continuation queries place on position i multiplied by the squared reconstruction error there, and replace the highest-scoring 20% with the consumer’s own entries, leaving 80% of the cache shared. The score depends on the consumer’s keys and values, so the resulting numbers bound what a cheaper estimator could achieve rather than describing a deployable scheme; we discuss this in Section 5.

3.6 Evaluation protocol

Three benchmarks are used. On ARC-Easy [22] we fine-tune a teacher for 10 epochs on 1000 multiple-choice questions until it memorizes the answer tokens, then test whether the mapped cache reproduces those decisions. On WikiText-2 [23] we fine-tune a language-modeling specialist for 3 epochs on 1000 training chunks and evaluate on 1000 held-out test sequences, each a 128-token prefix followed by 64 scored continuation tokens, for 64,000 tokens in total. For cross-specialist transfer we train a MathQA specialist on GSM8K [24] and a PythonCode specialist on an Alpaca-formatted Python instruction corpus [25], 3 epochs each from the same base, and map between them in both directions. Train and test splits are disjoint except in the ARC-Easy case, which is an in-sample retention probe by design.

Fidelity is measured against the consumer model running natively rather than against ground truth. Top- k agreement is the fraction of scored positions at which the native model’s argmax appears among our top k predictions. Confident Top-1 restricts attention to decisive positions, those where the native model’s top-1 to top-2 logit margin is at least 1.0. Top-5 overlap is the fraction of positions where our argmax falls within the native top-5. Logit cosine is computed over the full 50,257-dimensional logit vector, and the reported KL is $T^2 D_{\text{KL}}(P_{\text{native}} \| P_{\text{shared}})$ at $T = 2$. For free rollout we generate 32 tokens greedily without teacher forcing and report sequence exact match, meaning all 32 tokens coincide with the native trajectory, and survival length, the index of the first divergence.

4 Results

4.1 Task retention under a fully shared cache

Table 1 reports ARC-Easy accuracy when the specialist reads a cache produced entirely by the unadapted base model and stores nothing of its own.

Table 1: Memorized-task retention under a fully shared KV cache ($N = 1000$). The specialist consumes the base model’s cache through W_{map} and keeps no cache of its own.

Configuration	Accuracy	Specialist KV memory
Base GPT-2 (zero-shot)	26.20%	—
Fine-tuned teacher (upper bound)	99.90%	baseline
W_{map} shared cache	99.90%	100% saved

The mapped cache matches the teacher exactly. This is a retention probe and not a generalization result, since the teacher was trained on the same 1000 items; what it establishes is that a linear map can route the base model’s cache into the teacher’s memorized answers without measurable loss. The distance from the 26.20% base accuracy indicates that the cache is being genuinely re-interpreted rather than passed through.

4.2 Exactness, latency and memory scaling

Table 2 profiles one decoding step across all 12 layers as a function of context length, comparing native attention, the naive full-cache transform and the dual-projection form of Equation (1).

Table 2: Decode-step latency and KV memory scaling. Memory is the analytic fp16 footprint for two models sharing one context. Latency is a per-step micro-benchmark over 12 layers, averaged over 40 steps. The speedup column is the naive transform divided by the dual projection.

Context S	Unshared KV (MB)	Shared KV (MB)	Native attn. (ms)	Naive transform KW_k, VW_v (ms)	Dual-proj. QW_k^T, OW_v (ms)	Speedup
128	9.0	4.5	0.934	3.357	3.510	1.0×
512	36.0	18.0	0.929	3.336	3.483	1.0×
1024	72.0	36.0	0.937	3.367	3.442	1.0×
2048	144.0	72.0	0.942	3.352	3.435	1.0×
4096	288.0	144.0	1.412	5.464	3.432	1.6×
8192	576.0	288.0	2.778	11.137	3.496	3.2×
16384	1152.0	576.0	5.536	22.626	5.906	3.8×

The identity holds numerically: the largest absolute difference between the two computation orders is 0.00×10^0 in single precision, so the reformulation introduces no additional approximation on top of the map itself.

The scaling follows the analysis. The naive transform is flat below 2048 tokens, where kernel launches dominate, and grows linearly thereafter, from 3.35 ms at 2048 to 22.63 ms at 16,384. The dual projection remains between 3.43 and 3.51 ms out to 8192 and rises only at 16,384, which is where the 3.8×

Two aspects of this table should not be overread. At short contexts the projection is not free relative to native attention, costing 3.51 ms against 0.934 ms at $S = 128$, since the 24 additional projection kernels per step dominate an otherwise cheap attention. The overhead is launch-bound and would mostly vanish under fusion, but it is real as measured. At long context the comparison

that matters is against native attention, where the dual projection costs 5.906 ms against 5.536 ms, an overhead of 6.7% at 16,384 tokens.

A third comparison, which we have not run, is against simply re-prefilling the context with the consumer model. Re-prefill is one pass over S tokens and is paid once, whereas the dual projection is paid at every decoding step, so the two scale in opposite directions as the generation length G grows. Our measurements bound the per-step term at 0.37 ms for $S = 16384$ across twelve layers but say nothing about the prefill term, and at short contexts, where the projection adds roughly 2.6 ms per step, re-prefilling is almost certainly cheaper. Cache translation should pay for itself when prefixes are long and generations are short, but locating the crossover requires an end-to-end measurement we have not made. The memory argument below does not depend on this comparison.

The two memory framings used in this paper are consistent with each other. Table 2 counts two models serving one context, where sharing halves the total. Tables 1 and 3 count the consumer alone, which stores nothing, so the marginal saving per additional model is 100%. The producer’s cache exists in both accountings.

4.3 Full-vocabulary generative fidelity

Table 3 evaluates the WikiText-2 specialist reading a projected base-model cache over 64,000 held-out tokens.

Table 3: Generative fidelity and perplexity on the WikiText-2 test split (1000 sequences, 128-token prefix, 64 scored tokens, 50,257-way output). Agreement metrics are measured against the fine-tuned teacher running natively.

Configuration	PPL	Top-1	Top-2	Conf. Top-1	Top-5 ovl.	Logit cos	KL ($T=2$)
Base GPT-2 (zero-shot)	33.84	63.30%	76.90%	81.09%	85.61%	0.9441	1.6913
Fine-tuned teacher	31.10	100.00%	100.00%	100.00%	100.00%	1.0000	0.0000
W_{map} , 100% saved	31.30	93.35%	98.76%	99.77%	99.90%	0.9933	0.0287
Mixed cache, 80% saved	31.19	95.47%	99.36%	99.95%	99.98%	0.9970	0.0126

With no consumer-side cache at all, perplexity rises from 31.10 to 31.30, a relative increase of 0.6%, and the model reproduces the teacher’s exact token choice at 93.35% of positions. The residual disagreement is concentrated where the teacher is itself close to indifferent: on decisive tokens, those with a logit margin of at least 1.0, agreement is 99.77%, and across all positions 98.76% of teacher choices fall within the shared model’s top two. For reference, the base model agrees on 63.30% of positions with a KL of 1.69, against 0.0287 for the mapped cache, so the map is recovering behaviour that the base model on its own does not have.

Replacing the 20% of positions with the highest attention-weighted drift recovers roughly a third of the remaining Top-1 gap, reaching 95.47% while sharing 80% of the cache. The gain is modest relative to the cost of identifying those positions, which we discuss below.

4.4 Transfer between specialists

Table 4 covers the case the method is intended for, in which two independently fine-tuned specialists, neither of them the base model, hand a cache to one another.

Under teacher forcing the mapped cache closes most of the gap to native operation. Code $\rightarrow$ Math perplexity falls from 6.39 to 5.81 against a native 5.69, and Top-1 agreement rises from 87.61% to 93.74%. Untranslated transfer is not catastrophic in this protocol, which is worth noting: both

Table 4: Cross-specialist transfer with the target storing no cache of its own. “Raw” denotes the source model’s cache handed over untranslated. Teacher-forced metrics use 1000 held-out sequences; rollout is 32 greedy tokens without teacher forcing. Survival is the mean index of the first divergence from the target’s native trajectory, out of 32.

Transfer	Perplexity			Top-1 agreement		Sequence EM		Survival	
	Native	Raw	W_{map}	Raw	W_{map}	Raw	W_{map}	Raw	W_{map}
Math $\rightarrow$ Code	4.75	5.06	4.88	90.71%	93.90%	19.3%	25.7%	10.1	13.1
Code $\rightarrow$ Math	5.69	6.39	5.81	87.61%	93.74%	1.5%	10.7%	4.6	9.7

specialists descend from the same base, and teacher-forced evaluation resets the context at every position, so no single bad prediction can propagate.

Free rollout removes that protection and separates the two conditions much more clearly. In the Code $\rightarrow$ Math direction the raw cache reproduces the target’s full 32-token trajectory on 1.5% of sequences and diverges after 4.6 tokens on average, whereas the mapped cache reaches 10.7% and 9.7 tokens. The map therefore roughly doubles survival length and increases exact match sevenfold, but it does not eliminate divergence: 9.7 out of 32 remains far from reproducing the native trajectory. The asymmetry between the two directions, 13.1 against 9.7, suggests the code specialist’s representations are the easier target. Small per-token probability differences compound quickly under greedy decoding, and a linear map does not remove them.

5 Conclusion and Limitations

Fine-tuning displaces the key and value geometry of a shared base, and at GPT-2 scale enough of that displacement is linear that a head-wise 64×64 projection, roughly 1% of the backbone, allows one model to read another’s cache. Applying the map through the query and the output rather than the cache makes it exact and constant-time per decoding step, so one cache can serve several specialists without duplication and without re-prefilling. The projected cache preserves 93.35% of the teacher’s token choices and 99.77% of its decisive ones at a 0.6% perplexity cost, and it substantially slows the error compounding that otherwise destroys cross-model handoff under free generation.

Several limitations qualify these results. The most important is scale and architecture: every experiment uses GPT-2 small with multi-head attention and learned positional embeddings, whereas current models use grouped-query or latent attention, in which far fewer key and value parameters are available for a per-head map to exploit. Rotary embeddings deserve a specific note, because DeepSeek-V2 reports that RoPE blocks weight absorption [10]. That obstruction arises when the rotation is interposed between the two matrices being combined. In our setting the cache already holds rotated keys and W_k is learned on top of them, so nothing is interposed and Equation (1) continues to hold; what remains untested is whether a single position-independent W_k is a good fit in post-rotation space.

A second limitation concerns context length. Adapters are trained and evaluated at 128-token prefixes, while the latency results in Table 2 extend to 16,384 tokens using synthetic tensors, so we have no evidence that fidelity is maintained at those lengths. Relatedly, as discussed in Section 4.2, we do not measure the re-prefill baseline that the method ultimately competes against, and the regime in which translation is cheaper than recomputation is therefore not established.

The mixed-cache row should be read as optimistic. Its saliency score requires the consumer’s own keys and values for the prefix, which are precisely the tensors the method exists to avoid

computing, so the 80%-saved configuration is an upper bound rather than a deployable scheme; a practical version needs a drift estimator that does not consult the target model.

Two further costs are worth recording. One adapter is needed per ordered pair, so N specialists require $N(N - 1)$ of them; at 1.2M parameters each the storage is negligible but the training cost grows quadratically, and routing through the base model’s space as a pivot would reduce this to $2N$. And the dual projection, implemented as separate einsum calls, adds 6.7% over native attention at 16,384 tokens and considerably more at short contexts; folding W_k into the query projection and W_v into the output projection at load time should remove most of that, since both are weight-space compositions, but we have not implemented it.

Finally, teacher-forced agreement above 93% coexists with sequence exact match between 10% and 26%. Where a shared-cache path must be indistinguishable from the native one over long generations, a linear map by itself is not sufficient.

Code Availability

The implementation, including the adapter, the training procedures and all four benchmarks reported here, is available at <https://github.com/shahidchdry/kv-sharing>. All experiments use publicly available checkpoints and datasets and run under a fixed seed of 42.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [3] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [5] N. Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, pages 4895–4901, 2023.
- [7] W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In *Advances in Neural Information Processing Systems 37*, 2024.
- [8] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36*, pages 34661–34710, 2023.
- [9] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [11] I. Gim, G. Chen, S. Lee, N. Sarda, A. Khandelwal, and L. Zhong. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

- [12] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, pages 62557–62583, 2024.
- [13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [14] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [15] T. Mikolov, Q. V. Le, and I. Sutskever. Exploiting similarities among languages for machine translation. *arXiv preprint arXiv:1309.4168*, 2013.
- [16] K. Lenc and A. Vedaldi. Understanding image representations by measuring their equivariance and equivalence. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [17] Y. Bansal, P. Nakkiran, and B. Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems 34*, pages 225–236, 2021.
- [18] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning (ICML)*, 2019.
- [19] L. Moschella, V. Maiorca, M. Fumero, A. Norelli, F. Locatello, and E. Rodolà. Relative representations enable zero-shot latent space communication. In *International Conference on Learning Representations (ICLR)*, 2023.
- [20] M. Huh, B. Cheung, T. Wang, and P. Isola. Position: The Platonic representation hypothesis. In *International Conference on Machine Learning (ICML)*, pages 20617–20642, 2024.
- [21] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.
- [22] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [23] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.
- [24] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [25] Tarun. python_code_instructions_18k_alpaca. Hugging Face dataset, derived from sahil2801/code_instructions_120k, 2023. https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca.
- [26] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.